

第四课：Agent核心认知框架——从 ReAct 到自我反思的进化之路

01

课程主题

项目	内容
课程主题	Agent核心认知框架 —— 实时决策、策略蓝图、回路验证与偏差修正
课时安排	第2周第2次课（2小时）
学员基础	已完成第1-3课，掌握 Agent 基础、Function Calling 原理、LangChain 与 OpenAI 交互
教学目标	1. 深入理解 ReAct 模式及其在 Agent 实时决策中的核心作用 2. 掌握 Plan-and-Execute 策略蓝图的设计思想与实现 3. 理解 Self-Ask 回路验证机制 4. 学习 Thinking and Self-Reflection 偏差修正方法 5. 能够将不同认知框架应用到实际问题中

02

环境准备

- Python 3.11+ 环境
- 大模型 API 密钥（OpenAI / 国产模型）
- 已安装： `langchain` , `langchain-openai` , `openai` , `python-dotenv`

03

课程详情

第一部分：开场与上节回顾

1.1 【前言】

大家好，欢迎回到 AI Agent 全栈开发课程的第五课。

经过前几节课的训练，想必大家应该已经能写、会调用模型的 API 了。上节课我们更是深入到了 Function Calling 的底层，亲手用原生 SDK 模拟了两次 API 调用的完整流程——这个基础我们打得很扎实。

1.2 【上节课核心回顾】

下面，我们再来快速回顾一下上节课的三个关键点：

第一，Function Calling 的本质：它不是提示词工程的升级版，而是一种范式转换——模型从输出文本变成输出结构化指令。这就像一个人从‘只会说话’进化到‘会发号施令’。

第二，两次调用的必然性：第一次让模型决定‘用什么工具、填什么参数’；第二次让模型把工具返回的原始数据翻译成人类话。这也是 Agent 成本比普通对话高的原因。

第三，描述的重要性：工具的描述就是模型的说明书。描述越具体，模型调用越准确。‘获取天气’和‘当用户询问天气、温度、降水时调用’，效果天差地别。

上节课结束的时候，我们已经能用 Function Calling、LangChain 简单写出带多个工具的 Agent。Agent 的‘手脚’已经长出来了。

1.3 【引出本节课主题】

但是，大家有没有想过一个问题：**有了手脚，Agent 就能像人一样解决问题了吗？**

答案是：远远不够。这就像即使你给一个婴儿装上机械臂，它也还是不会做饭。

因为做饭这个事需要**规划**，比如：先洗菜、切菜、下锅……

而且炒菜时你还需要实时做**判断**，**比如**：火候够不够？盐放多了怎么办？

还需要**反思**：如果感觉这次做菜不在状态，下次我应该怎么改进等等？

同样的道理，一个真正的智能体，也不应该只能执行单步指令。它还要能够像人类一样：

- 面对复杂任务，先**思考**怎么做
- 然后**分步执行**，过程中不断**观察**结果
- 如果发现偏差，及时**修正**
- 甚至做完之后**自我反思**，下次做得更好。

上面描述的这些，就是我们今天要讲的核心——**Agent 的认知模式**。

1.4 【四种认知模式简介】

接下来，我们就要学习四种重要的认知模式，它们层层递进，构成了 Agent 从‘机械执行’到‘智能体’的持续进化。

第一种认知模式：ReAct 模式。也就是**思考-行动-观察**的循环。这是最基础的实时决策框架。模型每一步都在想‘我现在知道什么、下一步该做什么’，然后动手，看看结果，再继续。就好比你在修电脑：先想可能是内存问题，于是拔插内存条，观察有没有报警声，再想下一步。ReAct 模式让 Agent 能够动态应对未知情况。

第二种认知模式：Plan-and-Execute ——先规划后执行模式。如果你已经知道任务可以拆成固定步骤（比如‘查天气→计算→搜百科’），那就不需要每一步都让大模型思考，而是先生成一份计划，然后按计划执行。这样效率更高、成本更低。这个过程就像你执行出差任务前先列个清单，然后照着做。

第三种认知模式：Self-Ask——自问自答，回路验证模式。当模型不确定时，它会自己问自己：‘我需要知道X吗？’然后去查资料。这种模式特别适合需要多跳推理的知识问答。比如问‘北京和上海谁更

大？’，模型会先问‘北京人口多少？’，再问‘上海人口多少？’，然后比较。

第四种认知模式：Thinking and Self-Reflection——偏差修正与自我改进模式。做完一件事后，让模型自己评价：‘我做得好不好？有没有遗漏？’然后根据反馈修改。这就像写完代码后自己跑一下单元测试，发现 bug 再改。反思机制是提升Agent可靠性的关键。

需要知道的是，这四种模式并不是孤立的。一个成熟的 Agent 就像人类一样，不应该只具备一种认知能力，往往同时具备上述认知框架的多种能力。比如：用 ReAct 模式处理动态交互，用 Plan-and-Execute 模式执行固定流程，用 Self-Ask 模式分解复杂问题，然后再用 Reflection 模式自我纠错等等。

1.5 【课程结构预告与过渡】

今天课程我们共分为五个部分：

1. **ReAct 模式深度解析**：我们会介绍 ReAct 模式的原理、LangChain 实现以及手动实现。
2. **Plan-and-Execute 模式深度解析**：我们会介绍 Plan-and-Execute 模式的设计思想、LangGraph 实现、以及跟 ReAct 模式的对比情况。
3. **Self-Ask 回路验证模式**：我们会探索 Self-Ask 模式的自问自答机制、并如何与 ReAct 模式进行集成。
4. **Thinking and Self-Reflection 偏差修正与自我改进模式**：我们会探索这种模式如何实现自我评价与修正，并且给出代码示例。
5. **最后，我们会实现一个命理机器人**。这个实战小项目会把上面这四种模式融合起来，一起构建一个能算命、能反思的智能体。

同时，我们最后也会给大家布置一些作业，通过做具体的任务在实战中加深理解。

这节课知识是构建高级 Agent 的‘内功’。掌握了这些智能体认知模式，相信大家可以设计出更聪明、更可靠的智能体，而不是只会回答‘现在几点’的玩具。

第二部分：ReAct模式——思考-行动-观察的实时决策

2.1 ReAct 核心原理

2.1.1 【引入：从“一步到位”到“分步走”】

好，下面我们开始正式进入第一部分——**ReAct**，也就是我们非常熟悉的**思考-行动-观察**的实时决策模式。

ReAct 这个词，是 Reason 和 Action 的拼接，翻译过来就是‘思考-行动’。它的出处是 2022 年 Google Research 和 Princeton 联合发表的一篇论文。也就是这篇论文，可以说是 Agent 领域的奠基之作。

它的核心思想非常简单，但又极其深刻：**让模型在生成回答的同时，也能‘思考’下一步该做什么，然后采取行动，观察结果，再思考，循环往复。**

这个过程就像第一次学习做菜：你首先看菜谱思考，接着采取行动切菜，然后观察切得好不好，如果发现切太粗了，下次就切细一点。这个过程就是思考、行动和观察，并持续进行。整个过程你并不会指望执行一遍就能完美解决。

2.1.2 【ReAct三元组：Thought → Action → Observation】

ReAct 每一轮都由三个部分组成，我把它叫做‘三元组’：

第一，Thought（思考）：模型基于当前状态，用自然语言推理下一步应该做什么。这一步是透明的，你可以看到模型在想什么。

第二，Action（行动）：执行具体的动作。这个动作可以是调用一个工具（比如查天气、算数学），也可以是直接给出最终答案。

第三，Observation（观察）：观察行动的结果。如果是调用了工具，观察就是工具返回的数据；如果是直接回答，观察就是用户的反馈。

然后，模型把观察结果作为下一轮思考的输入，继续循环。

就像上面举的那个例子。当你在修电脑时，你先想‘可能是内存问题’，这就是思考，然后拔插内存条，这个动作就是行动，观察有没有报警声，这就是观察环节。如果不是内存内存，那有可能是显卡，这时就会进入下一轮思考……循环往复，使得每一步都有据可查。

2.1.3 【对比传统模型：一步到位 vs 分步走】

我们都知道，过去传统语言模型是**一步到位的**。你输入问题，模型直接输出答案。对于简单问题，这个过程没毛病。但如果问题很复杂，需要多步推理，比如‘北京和上海哪个城市人口更多？’，传统模型可能会瞎猜，或者只凭训练记忆进行回答，而模型训练的数据有可能是几年前的，不一定准。

这时，如果我们采用 ReAct 智能体，针对上面的问题，Agent 就会执行如下步骤。

1. **进行思考**: 它查询两个城市的人口数据。
2. **执行行动**: 它调用人口查询工具，参数进行赋值，比如： `city=北京` → 查询结果返回北京人口 2185万，这个返回结果就是**观察的结果**。
3. **再次进行思考**: 还需要上海的人口。
4. **再次行动**: 它调用人口查询工具，进行参数赋值，比如： `city=上海` → 查询结果返回上海人口 2428万，这个返回结果就是观察的结果。
5. **进入第三轮思考**: 经过比较，上海人口更多一些，可以回答。
6. **直接回答**: ‘上海人口更多’。

上面每一步都有明确的思考记录，从中，你可以看到模型具体是怎么一步步推理出来结果的。而且它调用的是实时工具，数据是准确的，不会用几年前的训练数据糊弄你。

上面的操作步骤，就是 ReAct 的核心价值：过程透明、结果可靠、可调试。

2.1.4 【强调：边想边做，不是线性执行】

这个过程中，ReAct 模式保持一个优点，那就是：**它可以在执行过程中动态调整策略。**

比如，你提出你的要求：‘查一下北京天气，如果下雨就提醒带伞，否则推荐一个户外活动’。

如果是传统程序，你就需要写 `if` 判断，而 ReAct 的智能体则会自动查天气，如果发现晴天，它就会给你推荐活动。如果发现下雨，它会自动改成提醒带伞。这种动态决策的弹性能力，是 if-else 代码做不到的，因为很多任务我们无法穷举它所有的可能性。

但有得就有失，ReAct 模式也有代价：它每一步都要调用一次模型。一个复杂任务可能走 5-10 步，成本就上去了。而且如果任务本身就是固定的，比如有个任务就是固定的三步流程，这样每步都让模型思考是非常浪费的。

所以，ReAct 这种模式只适合 **不确定性高、需要探索** 的任务；对于**可预见的固定流程**，ReAct 这种思考-行动-观察模式就不是特别合适了。

2.2 ReAct 在 LangChain 中的实现

2.2.1 【开场：框架帮你做了脏活】

ReAct 认知模式理论我们已经学习完了，下一步我们写个程序做到知行合一。

我们还是采用 LangChain 框架，至于这里为什么我们要用 LangChain 框架，因为如果是从零开始手动实现一个 ReAct 循环，还是挺麻烦的。我们需要自己写 while 循环、处理 `function_call`、管理消息历史等。好在 LangChain 框架的 `create_agent` 函数已经内置了 ReAct 循环，我们只需要提供工具，框架会自动帮你构建。

总之就是：框架帮我们实现了一切脏活。

因为大家已经有了上面几节课的基础，所以在流程上，我们只需要做三件事：

1. 定义工具（用 `@tool` 装饰器）
2. 初始化模型
3. 调用 `create_agent`，把工具传进去

然后框架就会非常自动化地构建 ReAct 循环。接下来，我们就来实现这个完整的例子。

2.2.2 【代码演示：查天气、算数学、搜百科】

这个例子，我们写一个能同时做三件事的 Agent：

1. 第一件事：查天气
2. 第二件事：算数学
3. 第三件事：搜百科

实现代码如下，我们一点一点来理解。

我们先创建 `4-cognition` 目录，然后执行 `python -m venv venv` 创建一个虚拟环境，因为我已经创建过了，所以这里我先通过 `source` 指令激活并进入该虚拟环境。

然后查看 `react.py` 代码。

代码首先导入依赖，并加载环境变量：

代码块

```
1 import os
2 from dotenv import load_dotenv
3 from langchain_openai import ChatOpenAI
4 from langchain_core.tools import tool
5 from langchain.agents import create_agent
6
7 load_dotenv()
```

然后定义三个工具，这三个工具分别是获取天气、计算算数表达式以及搜索wiki百科。

注意看每个工具的**文档字符串**——这是模型决定何时调用的关键。

代码块

```
1 @tool
2 def get_weather(city: str) -> str:
3     """获取指定城市的天气。参数city为城市名。"""
4     weather_db = {"北京": "晴 5-15°C", "上海": "阴 8-18°C"}
5     return weather_db.get(city, "未知城市")
6
7 @tool
8 def calculate(expression: str) -> str:
9     """计算数学表达式，如'2+3*4'。"""
10    try:
11        result = eval(expression, {"__builtins__": {}}, {})
12        return f"{expression} = {result}"
13    except:
14        return "计算错误"
15
16 @tool
```



```

17 def search_wiki(query: str) -> str:
18     """搜索维基百科获取信息。参数query为搜索词。"""
19     wiki_db = {"北京": "北京是中国的首都", "上海": "上海是中国的经济中心"}
20     return wiki_db.get(query, "未找到相关信息")

```

我们这里用模拟数据，实际你可以换成真实的 API 进行调用，大家如果闲得没事的话，下去可以自己实现。

接下来，我们初始化模型，创建 Agent：

代码块

```

1 model = ChatOpenAI(model="gpt-4", temperature=0)
2
3 agent = create_agent(
4     model=model,
5     tools=[get_weather, calculate, search_wiki],
6     system_prompt="你是一个助手，可以使用工具。回答要简洁。"
7 )

```

最后，给它一个复杂的任务，包含三个子任务：

代码块

```

1 response = agent.invoke({
2     "messages": [{"role": "user", "content": "北京天气怎么样？然后帮我算一下
3     15*8+20，最后查一下上海的信息。"}]
4 })
5 # 打印友好结果print("\n" + "="*50)print("🤖 最终回答:")print("="*50)
6 final_answer = response["messages"][-1].content
7 print(final_answer)

```

运行一下，你会看到类似输出：

代码块

```

1 (venv) ai@aideMacBook 4-cognition % python react.py
2 =====
3 最终回答：
4 北京的天气是晴，温度在5-15°C。15*8+20的结果是140。上海是中国的经济中心。

```

代码执行成功后截图如下：

```
(venv) ai@aideMacBook 4-cognition % !python
python react.py
/opt/homebrew/lib/python3.11/site-packages/langgraph/checkpoint/serde/encrypted.py:5: LangChainPendingDeprecationWarning: The default value of `allowed_objects` will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
  from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer

=====
最终回答：
北京的天气是晴， 温度在 5-15℃。15*8+20的结果是 140。上海是中国的经济中心。
```

该实例，模型自动规划了顺序：先查天气，再计算，最后查百科。你没有告诉它先做什么后做什么，它自己推理出来的。

2.2.3 【解释：背后发生了什么】

程序运行成功后，下面我们解释下这背后到底发生了什么？

从程序来看，`create_agent` 内部实际构建了一个 ReAct 循环，流程大致是这样的：

1. 模型首先收到用户消息，**进入思考状态**，确认用户问了三个问题，需要依次处理。先查天气。
2. 确认好先查天气后，立刻付诸行动，调用 `get_weather("北京")` 函数，获得结果：晴 5-15℃，这就是观察结果。
3. 模型再进行下一步思考——天气查完了，接下来要计算。
4. **于是再次采取行动**，调用 `calculate("15*8+20")` 函数，获得结果 ‘140’
5. 接着模型继续思考——需要查上海的信息。
6. **说干就干**，接着调用 `search_wiki("上海")` 函数，获得结果：‘上海是中国的经济中心’
7. 执行这一步后，马上接近尾声，**模型最后思考**：所有任务完成，整合回答。
8. **最终模型**直接输出最终答案。

后续我们会学习到 LangSmith，这样每一步的 Thought、Action、Observation，你都可以看到完整轨迹。这对于调试会非常有用。

2.2.4 【强调：依赖关系自动处理】

需要注意，这个例子中三个子任务并没有依赖关系，因此顺序不重要。但如果有依赖，比如‘查完天气后根据天气推荐穿衣’，模型也会自动处理顺序。

举个例子：比如‘北京天气怎么样？如果下雨就提醒带伞，否则推荐一个户外活动。’模型的行为是，会先查天气，拿到结果后，根据‘晴’或‘雨’来决定下一步是推荐活动还是提醒带伞。这种动态分支，ReAct 天然支持。

这就是 ReAct 的强大之处：模型不仅是按顺序执行工具，它还能根据观察结果动态调整后续行动。

2.3 手动实现 ReAct 循环

2.3.1 【开场：揭开黑盒】

刚才我们用 LangChain 的 `create_agent` 几行代码就实现了 ReAct Agent。但大家有没有想过：如果不用框架，我们自己怎么手动实现 ReAct Agent。

这里说明下，理解手动实现，至少有三个好处：

1. **调试时能定位问题**：模型为什么不调用工具？是不是 `function_call` 解析错了？
2. **定制化需求**：框架不支持的场景，你可以自己扩展。
3. **功利。面试时可以装X**：当你手撕 ReAct 循环，面试官会觉得你底子很厚。

下面，我们就来实现一个简化版 ReAct，其实这就是一层窗户纸，一捅就破。背后的原理非常简单，核心本质就是一个 `while` 循环。

2.3.2 【代码实现：手动ReAct】

我们依旧沿用刚才的三个工具，但这次我们不用 `create_agent`。

首先，定义工具的函数（模拟数据），然后构造函数定义（这里我们仍然采用 OPENAI API）。

为了节省时间，我直接写核心循环：

代码块

```
1  import openai
2  import json
3
4  def manual_react_agent(user_input, tools, max_steps=5):
5      messages = [{"role": "user", "content": user_input}]
```

```

6     step = 0
7     while step < max_steps:
8         # 调用模型
9         response = openai.ChatCompletion.create(
10             model="gpt-4",
11             messages=messages,
12             functions=tools, # 工具定义
13             function_call="auto"
14         )
15         message = response["choices"][0]["message"]
16
17         # 检查模型是否想调用工具
18         if message.get("function_call"):
19             func_name = message["function_call"]["name"]
20             args = json.loads(message["function_call"]["arguments"])
21
22             # 执行对应的真实函数
23             if func_name == "get_weather":
24                 result = get_weather(args["city"])
25             elif func_name == "calculate":
26                 result = calculate(args["expression"])
27             elif func_name == "search_wiki":
28                 result = search_wiki(args["query"])
29             else:
30                 result = "未知工具"
31
32             # 将模型的function_call消息和函数结果追加到消息列表
33             messages.append(message)
34             messages.append({
35                 "role": "function",
36                 "name": func_name,
37                 "content": str(result)
38             })
39         else:
40             # 没有工具调用, 返回最终答案
41             return message["content"]
42
43         step += 1
44
45     return "达到最大步数, 未完成"

```

我们解释下核心逻辑：

- 每次循环调用模型，传入当前消息列表。
- 如果模型返回 `function_call`，我们就解析出函数名和参数，调用真实函数，然后把结果以 `role="function"` 的消息追加回去。

- 然后继续循环。
- 如果模型不再返回 `function_call`，说明它准备输出最终答案，我们就返回这个答案。

运行成功后：

代码块

```
1 print(manual_react_agent("北京天气怎么样?", functions))
```

你会在控制台看到模型输出类似‘北京天气晴，5-15°C’模型返回结果。

2.3.3 【对比框架实现】

对比一下，手动实现我们需要大概30行核心代码，而 LangChain 的 `create_agent` 函数一行就搞定了。

框架帮我们处理了大量脏话，比如帮我们：

- 自动管理 `messages` 历史
- 自动解析 `function_call` 并执行函数
- 自动处理多步循环
- 自动处理错误重试等

但也只有真实手搓，你才会知道当模型不调用工具时，可能是 `function_call` 字段缺失；当参数提取错误时，可以检查 `arguments` 的JSON格式。这些以后都是调试的线索。

2.3.4 【强调：手动实现的局限】

当然，需要承认的是，手动实现也有局限：我们没有处理工具执行失败的情况，也没有限制最大步数（虽然加了 `max_steps` 但不够优雅），更没有记忆管理。所以，生产环境还是建议大家使用框架。

但作为学习，手动实现是最好的老师。 今天你跟着手写一遍，ReAct 就刻在你脑子里了。

好了，ReAct 模式咱们就先讲到这里。接下来我们进入第二种认知框架——**Plan-and-Execute**，看看它如何解决ReAct 的先天不足。

第三部分：Plan-and-Execute——策略蓝图与分步执行

3.1 设计思想：先规划后执行

3.1.1 【开场：ReAct的“代价”】

刚才我们已经讲了 ReAct 认知模式，我们知道虽然 ReAct 模式具备‘边想边做’的强大灵活性。但凡事都有代价。

ReAct 的代价是什么？那就是它**每一步都需要调用一次模型**。

想象一下，一个需要10步的任务，ReAct 就要调用最少10次模型。如果每步都用 GPT-4，成本不低，而且速度慢。更重要的是，很多任务其实是可以预先规划好的，不需要每步都让大模型思考‘下一步该干嘛’。

比如，你每天早上的例行公事：起床→刷牙→洗脸→吃早饭→出门。你不会每做一步都停下来想‘我接下来该干嘛’，因为这是固定流程。

对于这类**结构化、可预见**的任务，我们有一种更高效的认知模式——那就是 **Plan-and-Execute**。

3.1.2 【核心思想：规划与执行分离】

Plan-and-Execute 的核心思想是**先规划，后执行**。它分为两个明确的阶段：

第一阶段：规划（Plan）

模型接收用户的最终目标，生成一个步骤列表。这个列表可以是调用工具、执行子任务、或者查询信息。规划只做一次，生成一份‘蓝图’。

第二阶段：执行（Execute）

按照规划好的步骤列表，依次执行每一步。执行过程中，不一定需要每次都调用大模型——如果某一步只是简单的函数调用，可以直接执行；如果某一步需要决策，可以再调用模型。但总体上，模型调用的次数远少于 ReAct。

这里我们打个比方：ReAct 模式就像你一边问路一边开车，每到一个路口都要停下来想想；Plan-and-Execute 模式就像你先用导航规划好整条路线，然后照着开，不用每到一个路口都重新计算。

3.1.3 【优点与缺点】

Plan-and-Execute 模式的优点很突出：

1. **效率高**：规划一次，执行多次，减少模型调用。
2. **成本低**：可以用便宜的小模型做规划，执行阶段甚至可以不用模型。
3. **可预测**：步骤列表清晰，便于调试和监控。

当然 Plan-and-Execute 模式缺点也很明显：

1. **不够灵活**：如果规划出错，或者执行过程中出现意外情况（比如某个工具返回了错误），整个计划可能失败，需要重新规划。
2. **不适合探索性任务**：对于需要动态调整的任务（比如‘帮我研究一下最新的AI趋势’），你没法提前规划所有步骤。

由此可知，Plan-and-Execute 模式的典型应用场景包括：数据ETL流程、批量报告生成、定时任务、固定的业务流水线。在这些场景下，Plan-and-Execute 模式是首选。

3.1.4 【与ReAct的互补关系】

所以说，Plan-and-Execute 模式不是要替代 ReAct 模式，它们的关系是**互补关系**。在实际应用中，你完全可以混合使用：

1. 你可以先用 Plan-and-Execute 快速执行固定流程。
2. 如果某一步执行失败或遇到异常，再切换到 ReAct 模式动态处理。

就像自动驾驶。在高速路段我们可以用定速巡航（Plan-and-Execute），而遇到复杂路况则切换回人工驾驶（ReAct）。

接下来，我们看看如何用 LangGraph 实现 Plan-and-Execute。

3.2 实现方案：使用 LangGraph 构建 Plan-and-Execute

3.2.1 【开场：为什么选 LangGraph？】

要实现 Plan-and-Execute，我们需要一个能管理状态、支持多步骤流程的框架。LangGraph 就是完美选择——它天然支持定义多个节点，还能在节点之间灵活跳转，你可以把它类比 Dify，或者是 Coze。

下面我们将构建一个三节点的图：

1. 规划节点 (planner)：生成步骤列表。
2. 执行节点 (executor)：按顺序执行每一步。
3. 最终节点 (finalizer)：汇总结果。

如果执行中遇到错误，我们还可以加入‘重规划节点’，但为了方便理解，我们今天先做个基础版。

3.2.2 【定义状态结构】

首先，我们定义状态。状态会在节点之间传递，比如保存当前计划、已执行的步骤等。

代码块

```
1 from langgraph.graph import StateGraph, END
2 from typing import TypedDict, List
3
4 class PlanExecuteState(TypedDict):
5     input: str          # 用户输入
6     plan: List[str]     # 步骤列表，如['查北京天气', '计算15*8+20']
7     past_steps: List[tuple] # 已执行的步骤和结果，如[('查北京天气', '晴'), ...]
8     response: str       # 最终输出
```

上面代码中的状态结构很清晰：`plan` 是待办清单，`past_steps` 是已完成事项清单。

3.2.3 【规划节点：生成步骤列表】

下一步，我们规划节点，该节点作用是把用户输入拆解成步骤。我们使用一个简单的 Prompt 提示词让模型帮我们生成编号列表。

代码块

```
1 def plan_step(state: PlanExecuteState):
2     """规划节点：用模型生成步骤，但要求使用工具"""
3     prompt = f"""分析以下任务，生成具体步骤。每个步骤必须是以下格式之一：
4     - "查天气:城市名" （如：查天气:北京）
5     - "计算:表达式" （如：计算:15*8+20）
6     任务: {state['input']}
7     只输出步骤列表，每行一个，不要解释。"""
8     response = model.invoke(prompt)
9     lines = response.content.strip().split('\n')
```



```

10     # 清理步骤
11     plan = [line.strip().lstrip('0123456789.- ') for line in lines if
line.strip()]
12     return {"plan": plan}

```

比如输入‘帮我查北京天气，然后计算15*8+20，最后总结’，模型可能生成：

代码块

```

1    1. 查询北京天气
2    2. 计算15*8+20
3    3. 总结以上结果

```

该函数作用是：在规划节点提取出 ['查询北京天气', '计算15*8+20', '总结以上结果'] 完整规划。

3.2.4 【执行节点：按顺序执行】

接下来，我们开始按顺序执行“执行节点”。执行节点每次取上一步 `plan` 中的第一个步骤，并执行它，执行完毕后，然后移除该步骤，最后重复直到 `plan` 为空。

代码块

```

1  def execute_step(state: PlanExecuteState):
2      """执行节点：解析步骤并调用真实工具"""
3      plan = state["plan"]
4      past_steps = state.get("past_steps", [])
5
6      if not plan:
7          return {"response": "所有步骤执行完成"}
8
9      step = plan[0]
10     print(f"🔧 执行步骤: {step}")
11
12     # 解析步骤并调用对应工具
13     result = ""
14     if "查天气" in step or "天气" in step:
15         # 提取城市名（简单解析）
16         city = step.split(":")[-1] if ":" in step else "北京"
17         result = get_weather(city)
18         print(f"    结果: {city}天气 = {result}")
19
20     elif "计算" in step:

```

```

21         # 提取表达式
22         expr = step.split(":")[-1] if ":" in step else step.replace("计算", "")
23         result = calculate(expr)
24         print(f"    结果: {expr} = {result}")
25
26     else:
27         # 其他步骤用模型处理
28         resp = model.invoke(f"完成任务: {step}")
29         result = resp.content
30
31     past_steps.append((step, result))
32
33     return {"past_steps": past_steps, "plan": plan[1:]}

```

注意：执行时我们把 `past_steps` 作为上下文传给模型，让模型知道之前已经做了什么。比如执行‘总结以上结果’时，模型能看到前面两步的结果。

3.2.5 【条件边与最终节点】

我们需要一个条件函数，该条件函数的作用是：决定执行完后是继续执行下一步，还是进入最终节点。

代码块

```

1  def should_continue(state: PlanExecuteState):
2      if state["plan"]:
3          return "execute"
4      else:
5          return "finalize"

```

最终节点负责把 `past_steps` 格式化最终输出：

代码块

```

1  def finalize(state: PlanExecuteState):
2      """总结节点"""
3      summary = "执行结果: \n" + "\n".join([
4          f"{i+1}. {step} -> {result}"
5          for i, (step, result) in enumerate(state["past_steps"])
6      ])
7      return {"response": summary}

```

接下来，我们开始构建图：

代码块

```
1  # ===== 构建图 =====
2  load_dotenv()
3  model = ChatOpenAI(model="gpt-4", temperature=0)
4
5  graph = StateGraph(PlanExecuteState)
6  graph.add_node("planner", plan_step)
7  graph.add_node("executor", execute_step)
8  graph.add_node("finalizer", finalize)
9
10 graph.set_entry_point("planner")
11 graph.add_edge("planner", "executor")
12 graph.add_conditional_edges("executor", should_continue, {"execute":
    "executor", "finalize": "finalizer"})
13 graph.add_edge("finalizer", END)
14
15 app = graph.compile()
```

然后我们调用 graph，并调用大模型推进整个流程执行：

代码块

```
1  result = app.invoke({"input": "帮我查北京天气，然后计算15*8+20，最后总结"})
2  print("\n" + "="*50)
3  print(result["response"])
```

程序执行成功后，会输出类似如下的信息：

代码块

```
1  🔧 执行步骤：查天气:北京
2      结果：北京天气 = 晴 25°C
3  🔧 执行步骤：计算:15*8+20
4      结果：15*8+20 = 140
5
6  =====
7  执行结果：
8  1. 查天气:北京 -> 晴 25°C
9  2. 计算:15*8+20 -> 140
```

3.3 对比ReAct与Plan-and-Execute

3.3.1 【对比表格】

好，接下来我们做个简单总结。我们再从多个维度，详细对比下 ReAct 模式和 Plan-and-Execute 模式：

维度	ReAct	Plan-and-Execute
决策方式	动态，每步都思考	静态，预先规划
模型调用次数	每步一次，任务步骤数=调用次数	规划一次 + 必要时执行调用，通常远少于 ReAct
适用场景	不确定性高、需要探索的任务	结构化、可预见的任务
灵活性	高，可随时调整	低，计划出错需重规划
成本	高（尤其长任务）	较低
速度	慢（串行调用模型）	快（可并行执行无依赖步骤）
典型应用	智能客服、研究助手、动态决策	数据流水线、报告生成、批量处理

我们逐条解读：

- 决策方式：ReAct 是‘边走边看’，Plan-and-Execute 是‘看好了再走’。
- 模型调用次数：ReAct 执行N步需要N次模型调用（可能更多，因为思考本身也是调用）。Plan-and-Execute 通常只需要1次规划，执行时可以用轻量级函数。
- 灵活性：ReAct 可以在执行中发现新信息后改变计划；Plan-and-Execute 如果第一步就错了，后面全错。
- 成本：一个 10 步的 ReAct 任务，用 GPT-4 可能花几块钱；Plan-and-Execute 可能只花几毛钱。
- 速度：ReAct 串行执行，Plan-and-Execute 可以并行执行没有依赖的步骤（比如同时查两个城市的天气）。

3.3.2 【混合使用策略】

在实际应用中，没有绝对最好的模式。聪明的做法是混合使用，下面是总结出来的三条实现策略：

策略一：Plan-and-Execute 为主，ReAct 为辅

先用 Plan-and-Execute 生成计划并执行。如果某一步执行失败（比如工具返回错误），就切换到 ReAct 模式，让模型动态处理这个异常，然后继续执行剩余计划。

策略二：ReAct 生成计划，Plan-and-Execute 执行

先用 ReAct（或专门的规划模型）生成计划，然后把计划交给 Plan-and-Execute 执行。这样既利用了 ReAct 的灵活性来制定好计划，又利用了 Plan-and-Execute 的高效执行。

策略三：分级规划

对于超长任务，可以分层规划：高层用 Plan-and-Execute 制定大的阶段，每个阶段内部用 ReAct 动态执行。这就像公司战略（高层计划）和战术执行（底层动态调整）的关系。

3.3.3 【总结与过渡】

好，Plan-and-Execute 我们就讲到这里。我们着重解释了它的设计思想、并尝试用 LangGraph 框架进行了实现，最后也和 ReAct 模式进行了多维度的对比。

可以简单记住这一句话：ReAct 模式适合探索，而 Plan-and-Execute 模式适合编排。聪明的 Agent 应该懂得在两者之间自由切换。

接下来，我们将要学习第三种认知框架——Self-Ask 模式，我们将看到模型如何通过自问自答来分解复杂问题。

第四部分：Self-Ask——回路验证与知识检索

4.1 Self-Ask 原理

4.1.1 【引入：当模型需要“查资料”时】

上面我们讲了 ReAct 模式和 Plan-and-Execute 模式，ReAct 负责动态决策，Plan-and-Execute 负责高效执行。不知大家有没有发现，其实这两个框架都有一个隐含假设：**模型非常清楚知道自己该做什么。**

但现实中，模型经常面临‘我不知道’的情况。比如，你问‘北京和上海哪个城市人口更多？’。模型如果只靠训练数据，可能记得2019年的数字，但2026年的最新数据呢？它不知道。那怎么办？——它需要去查。

这就是 Self-Ask 要解决的问题。它的名字很形象：**自己问自己**。模型通过自问自答的方式，把一个大问题拆解成多个小问题，然后逐个去查，最后综合答案。

4.1.2 【核心流程：拆解子问题】

为了深入了解 Self-Ask 模式，我们首先来看一下 Self-Ask 的核心流程，它的核心流程总共分三步：

1. **第一步拆解**：模型将原始问题分解成一系列子问题（Follow-up Questions）。这些子问题通常可以通过检索工具（如搜索引擎、数据库、API）直接回答的。
2. **第二步检索**：对每个子问题，调用外部工具获取答案。
3. **最后一步综合**：模型根据所有子问题的答案，推理出原始问题的最终答案。

我们还是举上面这个例子，比如问‘哪个城市更大，北京还是上海？’。Self-Ask 会这样操作：

- **子问题1**：北京的人口是多少？
- **子问题2**：上海的人口是多少？
- 然后比较两个数字。

这种情况下，模型不会直接回答‘北京更大’或‘上海更大’，而是先查数据。每个子问题都可以通过搜索工具（比如调用API）来获得准确答案。

这种模式相当于在模型内部建立了一个**问答回路**：模型先问自己‘我需要知道什么’，然后去找答案，拿到后再继续问自己‘然后呢’。每一步都有据可查，而不是凭空猜测。

上面这个流程就是 Self-Ask 模式。

4.1.3 【Self-Ask vs ReAct】

不知道大家听到这里会不会有点晕，有没有觉得 Self-Ask 模式跟上面我们介绍的 ReAct 模式很像。

确实有点，那么，Self-Ask 模式和 ReAct 模式到底有哪些区别呢？

- **ReAct** 是‘思考-行动-观察’的循环。模型思考后，直接采取行动（比如调用工具），然后观察结果，再思考。它强调的是**怎么做**——动态地与环境交互。
- **Self-Ask** 是‘自问-自答’的循环。模型先拆解问题，然后对每个子问题去检索答案，最后综合。它强调的是**要查什么**——结构化地分解知识需求。

打个比方：

- ReAct 像是一个侦探，一边调查一边想‘下一步该去哪里找线索’。
- Self-Ask 像是一个学者，先把大问题写成‘我需要知道A、B、C’，然后去图书馆查资料，回来再写论文。

简单总结就是：ReAct 重探索；Self-Ask 重检索。

4.1.4 【适用场景】

Self-Ask 最适合以下场景：

- **事实性问答**：需要最新或特定领域知识的问题，比如‘2025年诺贝尔和平奖得主是谁？’。
- **多跳推理**：需要多个步骤才能得出结论的问题，比如‘张三的导师的学生中，谁获得了图灵奖？’。
- **比较类问题**：需要对比多个实体的属性，比如‘哪款手机性价比更高？’。

在 Agent 开发中，Self-Ask 通常和检索工具（如搜索引擎、向量数据库）配合使用。模型负责拆解，工具负责提供事实。这样既发挥了模型的推理能力，又避免了模型编造答案。

下面，我们就看看如何使用 LangChain 框架，手动实现一个 Self-Ask 实例。

4.2 在 LangChain 中实现 Self-Ask

4.2.1 【手动实现Self-Ask链】

首先，我们定义 Prompt 模板：

代码块

```
1  from dotenv import load_dotenv
2  from langchain_core.prompts import PromptTemplate
3  from langchain_openai import ChatOpenAI
4
5  # 加载环境变量
6  load_dotenv()
7
8  # 定义模板
9  template = """
10  问题: {question}
11  请将这个问题分解成需要回答的子问题，用'子问题1: '、'子问题2: '等格式列出。
12  然后依次回答每个子问题，最后给出最终答案。
13
```

```

14 格式：
15 子问题1： ...
16 答案1： ...
17 子问题2： ...
18 答案2： ...
19 最终答案： ...
20 """
21
22 # 初始化模型和提示
23 model = ChatOpenAI(model="gpt-4", temperature=0)
24 prompt = PromptTemplate.from_template(template)
25
26 # 新版写法：使用管道符 | 组合链
27 chain = prompt | model
28
29 # 执行
30 question = "北京和上海哪个城市人口更多？"
31 result = chain.invoke({"question": question})
32
33 # 打印结果
34 print(result.content)

```

运行后，模型可能输出：

代码块

```

1 子问题1：北京的人口数量是多少？
2 答案1：根据2020年的数据，北京的常住人口为21.89 million。
3
4 子问题2：上海的人口数量是多少？
5 答案2：根据2020年的数据，上海的常住人口为24.87 million。
6
7 最终答案：上海的人口数量比北京多。

```

代码执行成功后截图如下：

```

(venv) ai@aideMacBook 4-cognition % python saa.py
子问题1：北京的人口数量是多少？
答案1：根据2020年的数据，北京的常住人口为21.89 million。

子问题2：上海的人口数量是多少？
答案2：根据2020年的数据，上海的常住人口为24.87 million。

最终答案：上海的人口数量比北京多。

```


我们需要注意：这里模型回答子问题时依赖的是自己的内部知识（训练数据），不是真正的实时检索。如果要获取最新数据，我们可以在每个子问题后插入真实的工具调用。

4.2.3 【优势与局限】

Self-Ask 的优势是**结构化**和**可追溯**。你能看到模型为了回答问题，问了哪些子问题，答案是什么。这在金融、医疗等需要审计的场景非常有用。

有得就有失，Self-Ask 模式的局限同样也很明显：

1. 模型拆分子问题的质量直接影响最终答案。如果拆解错了，答案就偏了。
2. 对于需要常识推理的问题，Self-Ask 可能过度拆解，显得冗余。
3. 如果每个子问题都要调用一次模型或 API，成本会上升。

好了，Self-Ask 我们就讲到这里。下面我们进入第四种认知框架，也是最后一种认知框架——**Thinking and Self-Reflection（思考与自省模式）**，该模式可以让 Agent 能够自我评价和修正。

第五部分：Thinking and Self-Reflection——偏差修正与自我改进

5.1 为什么需要自我反思？

5.1.1 【引入：Agent也会犯错】

上面我们学习了 ReAct、Plan-and-Execute、Self-Ask，这些框架模式可以让 Agent 具备动态决策、高效执行、知识检索的能力。但是，即使有了这些，Agent 仍然会犯错。

比如：常见的错误包括：

- **调用错误的工具：**用户问‘今天冷不冷？’，模型却调用了股票查询工具。
- **参数提取错误：**用户说‘帮我查一下北京明天的天气’，模型却提取了‘今天’。
- **逻辑错误：**模型得出结论的过程有漏洞，比如比较人口时用了不同年份的数据。
- **遗漏关键信息：**模型只回答了部分问题，比如用户问‘天气和温度’，模型只给了天气。
- **陷入死循环：**模型反复调用同一个工具，永远无法收敛。

当出现这些问题怎么解决？我们知道，人类犯错后，解决错误的关键能力之一是**反思**。做完一件事后，我们会回头检查：有没有更好的方法？有没有遗漏？有没有错误？然后根据检查结果修正。

对于 Agent，我们同样可以引入**自我反思**机制。它让 Agent 能够评估自己的输出，发现偏差，然后修正。

5.1.2 【反思的典型流程】

反思模式通常包含三个步骤：

1. **执行**：Agent 完成一个任务，得到初步结果。这个结果可能来自 ReAct 循环的最终输出，也可能来自 Plan-and-Execute 的某个阶段。
2. **评价**：调用一个‘反思模型’来评估结果是否符合预期。评估的维度可以包括：完整性（是否回答了所有子问题）、准确性（事实是否正确）、逻辑一致性（推理是否自治）、格式要求（是否按要求输出）等。这个反思模型可以是**同一个模型**（用不同的提示），也可以是**另一个独立的模型**。
3. **修正**：如果评估发现不足，生成修正意见，然后让 Agent 重新执行，或者直接修改结果。修正后可以再次进入评价环节，形成一个迭代循环。

这个过程就像你写完代码后自己跑单元测试（评价），发现 bug 就修改（修正），然后再跑测试，直到通过。

这个循环可以重复多次，直到结果满足质量要求或达到最大迭代次数。

5.1.3 【反思 vs 审计Agent】

注意，反思机制与我们之前提过的‘审计Agent’有相似之处，但也有区别：

- **审计Agent**：通常是一个**独立的**Agent，专门负责检查其他Agent的输出。它和主 Agent 是分离的，可以并行工作。优点是客观、不会‘自我感觉良好’；缺点是增加了系统复杂度和成本。
- **自我反思**：是**同一个 Agent** 对自己的输出进行评价和修正。它不需要额外的 Agent，但需要模型具备**元认知能力**——即‘思考自己的思考’。优点是轻量、简单；缺点是模型可能‘护短’，不愿意承认自己的错误。

在实际系统开发中，我们可以进行两者结合：让主 Agent 先自我反思，如果还不行，再交给审计 Agent 二次检查。

5.1.4 【应用场景与小结】

在真正实操之前，我们先了解下反思机制常用在哪些场景。经过实践，我们发现反思模式经常在以下场景特别有用：

- **代码生成**：Agent 写完代码后，自己运行测试，根据错误信息修改。
- **数据分析**：Agent 生成图表后，检查轴标签、标题、数据范围是否正确。
- **规划**：Agent 生成计划后，自己检查是否有遗漏步骤或依赖错误。
- **文本生成**：Agent 写文章后，检查是否跑题、语法错误、格式问题。

反思是提升 Agent 可靠性的关键。没有反思，Agent 就像一个只会闷头做事、从不回头看的工人；有了反思，它就变成了一个会自我改进的智能体。

接下来，我们就通过一个简单的例子，演示如何实现自我反思。

5.2 实现一个简单的反思Agent

5.2.1 【案例：短文生成与修改】

这个例子就是我们演示反思的完整流程：让 Agent 写一段关于某个主题的短文，然后自己评价并修改。这个例子虽然简单，但包含了反思的核心要素。

首先，我们初始化模型：

代码块

```
1  from dotenv import load_dotenv
2  from langchain_core.prompts import PromptTemplate
3  from langchain_openai import ChatOpenAI
4
5  # 加载环境变量
6  load_dotenv()
7  model = ChatOpenAI(model="gpt-4", temperature=0)
```

第一步：我们先定义一个生成短文的链，要求不超过50字。

代码块

```
1  generate_prompt = PromptTemplate.from_template("写一段关于{topic}的简短介绍，不超过50字。")
2  generate_chain = generate_prompt | model
3
4  topic = "人工智能"
5  first_draft = generate_chain.invoke({"topic": topic}).content
```

```
6 print("初稿: ", first_draft)
```

我们执行代码，程序可能的输出是： 人工智能是模拟人类智能的理论和应用，包括机器学习、自然语言处理等。

第二步：自我反思（评价）

紧接着，我们进入第二步，我们让模型评价这段文本的质量，指出不足之处并提出修改建议。

代码块

```
1 reflect_prompt = PromptTemplate.from_template("""评估以下文本的质量，指出不足之处，  
    并提出修改建议。  
2 文本: {text}  
3 不足: """)  
4 reflect_chain = reflect_prompt | model  
5 feedback = reflect_chain.invoke({"text": first_draft}).content  
6 print("反馈: ", feedback)
```

执行代码，程序可能给出的反馈是： 文本太简短，没有具体例子。建议加入一个实际应用场景，比如“人脸识别”或“语音助手”。

第三步：根据反馈修改

然后，我们来到了第三步，模型会根据反馈进行修改。

代码块

```
1 improve_prompt = PromptTemplate.from_template("""根据以下反馈修改原文。  
2 原文: {text}  
3 反馈: {feedback}  
4 修改后的版本: """)  
5 improve_chain = improve_prompt | model  
6 final_draft = improve_chain.invoke({"text": first_draft, "feedback":  
    feedback}).content  
7 print("终稿: ", final_draft)
```

执行代码，程序执行结果如下：

```
(venv) ai@aideMacBook 4-cognition % python tasr.py
初稿： 人工智能（AI）是计算机科学的一个分支，它试图理解和制造智能机器，特别是智能计算机程序。AI的目标包括学习、推理、知识表示、规划、导航、感知和自然语言处理等。
反馈： 1. 文本的内容虽然准确，但表述过于简单和公式化，缺乏深度和详细的解释。读者可能无法从这段描述中完全理解人工智能的含义和应用。
2. 文章没有引用任何资料或者数据支持其观点，缺乏说服力。
3. 文本的结构和语言表述过于平淡，缺乏吸引力。

修改建议：
人工智能（AI）是计算机科学的一个重要分支，它不仅试图理解智能机器的工作原理，更进一步地尝试制造出能模拟人类思维的智能计算机程序。AI的目标领域广泛，包括但不限于：学习能力，即让机器能自我学习，不断优化改进；推理能力，使机器能逻辑推断，解决复杂问题；知识表示，让机器能理解并储存知识；规划和导航，使机器能制定有效计划并自我导航；感知能力，提高机器感知周围环境的能力；以及自然语言处理，让机器能理解和使用人类语言。
终稿： 人工智能（AI），作为计算机科学的一个重要分支，它的目标远超过了对智能机器的简单理解和制造。AI尝试制造能模拟人类思维的智能计算机程序，这是一项深入人类思维核心的挑战。AI的目标领域多样且广泛，其中包括：学习能力，它是指让机器有能力自我学习并且不断优化改进，以适应不断变化的环境；推理能力，使机器具备逻辑推断能力，从而解决复杂问题；知识表示，让机器能理解并储存知识，实现知识的积累；规划和导航，使机器能制定有效计划并自我导航，以达成特定目标；感知能力，提高机器对周围环境的感知能力，实现与环境的交互；以及自然语言处理，使机器能理解和使用人类语言，实现人机交流。这些目标领域的实现，都需要科研人员深入研究，丰富的数据支持，并且需要不断的试错和优化。
```

以上这个三步流程，就是模型的自我反思的完整实现。

5.2.2 【反思的代价与平衡】

但我们必须明白，反思机制虽然提升了可靠性，但同样付出的代价也很明显：那就是**额外的模型调用**。因为每反思一轮，就要调用 1-2 次模型（评价+可能的修改）。对于高准确率要求的场景（如医疗、金融、代码生成），这是值得的；但如果对于一般闲聊的场景，这样的方式可能得不偿失。

所以，在实践中，我们建议采用‘按需反思’策略：

- 对于简单任务（如查天气），不反思。
- 对于中等复杂任务（如写一段代码），反思一轮。
- 对于高价值任务（如生成合同、分析财报），反思多轮，甚至引入人工审核。

第六部分：综合实战——命理机器人

6.1 场景介绍

6.1.1 【引入：融合四种认知框架】

上面四种智能体认知模式的理论和实践我们了解一些后。

下一步我们开发一个小项目，融合这四种认知框架。这个小项目就是命理机器人。

这个机器人的场景我们介绍下，它可以根据用户的出生信息，推算星座、生肖、运势，并给出个性化建议。它看似简单，但涉及多步推理、信息收集、记忆管理和质量校验，非常适合作为我们学习的综合实战场景。

6.1.2 【需求拆解】

我们首先来拆解一下该项目的功能需求，经过梳理，它一共包含6个主要的功能需求，分别如下：

1. **信息收集**：用户可能只说‘帮我算算运势’，没给生日。机器人需要主动询问。这适合 **ReAct** 模式，因为需要动态交互。
2. **星座/生肖计算**：根据生日计算星座和生肖。这是确定性规则，不需要调用模型。适合 **Plan-and-Execute** 中的执行节点，直接调用函数。
3. **运势生成**：可以基于星座随机生成，或调用第三方API。这里我们用随机模拟，放在执行节点。
4. **建议生成**：结合运势和用户的历史信息（如之前说过‘最近工作压力大’），生成个性化建议。这需要模型推理，可以放在 ReAct 的 Thought 中或作为单独的执行步骤。
5. **多轮记忆**：用户连续提问（如‘那明天的运势呢？’），需要记住之前的星座、偏好等。这里我们用 LangGraph 框架的 `StateGraph` + `checkpoint` 进行实现。
6. **自我反思**：如果最终回答缺少星座、生肖或运势，机器人应该能检测到并重新生成。我们加入一个反思节点。

6.1.3 【技术选型与架构设计】

需求我们介绍完毕了，下面我们介绍下该项目将采用的架构模式，那就是**混合架构，具体实现流程为**：

1. 主流程用 **Plan-and-Execute**：先规划步骤（解析生日→算星座→算生肖→运势→建议），然后执行。
2. 执行过程中，如果需要用户补充信息，切换到 **ReAct** 动态交互。
3. 在生成建议后，加入 **Self-Ask** 检查是否遗漏关键信息（比如用户之前提过的偏好）。
4. 最后我们使用**反思**机制检查回答是否完整。

在技术实现上，我们选择 **LangGraph** 作为主框架，因为它天然支持复杂的状态管理和多节点流程。

我们将定义以下节点：

1. `collect_info`（第一个节点）：收集用户信息，该节点我们使用 ReAct 风格
2. `planner`（第二个节点）：规划节点。用来生成步骤计划
 - `executor`（第三个节点）：执行节点。用来依次执行步骤，也就是调用工具函数
 - `reflector`（第四个节点）：反思答案节点，确认是否切题，并没有大的疏忽。

- `finalizer`（第五个节点）：结束节点。用来输出最终结果。

下面我们开始逐步编码，进行实现。

6.2 定义工具与规划

6.2.1 【定义工具函数】

首先，我们先定义命理计算所需的所有本地工具函数。这些函数不调用模型，只是纯逻辑。我们用 `@tool` 装饰器包装，以便在 LangChain 框架中使用。

代码块

```
1  # ===== 工具定义 =====
2  @tool
3  def get_zodiac(birth_date: str) -> str:
4      """根据出生日期 (YYYY-MM-DD) 返回星座"""
5      try:
6          date = datetime.strptime(birth_date, "%Y-%m-%d")
7          month, day = date.month, date.day
8          zodiac_dates = [
9              ((3, 21), (4, 19), "白羊座"), ((4, 20), (5, 20), "金牛座"),
10             ((5, 21), (6, 21), "双子座"), ((6, 22), (7, 22), "巨蟹座"),
11             ((7, 23), (8, 22), "狮子座"), ((8, 23), (9, 22), "处女座"),
12             ((9, 23), (10, 23), "天秤座"), ((10, 24), (11, 22), "天蝎座"),
13             ((11, 23), (12, 21), "射手座"), ((12, 22), (1, 19), "摩羯座"),
14             ((1, 20), (2, 18), "水瓶座"), ((2, 19), (3, 20), "双鱼座")
15         ]
16         for (start_m, start_d), (end_m, end_d), name in zodiac_dates:
17             if (month == start_m and day >= start_d) or (month == end_m and
18                 day <= end_d):
19                 return name
20             return "摩羯座"
21         except:
22             return "未知"
23  @tool
24  def get_chinese_zodiac(birth_date: str) -> str:
25      """根据出生日期返回生肖"""
26      try:
27          year = datetime.strptime(birth_date, "%Y-%m-%d").year
28          zodiacs = ["猴", "鸡", "狗", "猪", "鼠", "牛", "虎", "兔", "龙", "蛇",
29                     "马", "羊"]
```

```

29         return zodiacs[year % 12]
30     except:
31         return "未知"
32
33 @tool
34 def get_daily_luck(zodiac: str, day_offset: int = 0) -> str:
35     """根据星座返回运势 (day_offset=0今天, 1明天) """
36     luck_options = ["大吉", "中吉", "小吉", "平"]
37     # 用日期做种子, 保证同一天结果一致, 不同天不同
38     base_date = datetime.now().date()
39     target_date = base_date + timedelta(days=day_offset)
40     random.seed(f"{zodiac}_{target_date}")
41     luck = random.choice(luck_options)
42     random.seed() # 重置种子
43     day_str = "今日" if day_offset == 0 else "明日"
44     return f"{day_str}运势: {luck}"
45
46 @tool
47 def generate_advice(zodiac: str, chinese_zodiac: str, luck: str) -> str:
48     """生成个性化建议"""
49     prompt = f"用户星座{zodiac}, 生肖{chinese_zodiac}, {luck}。给一条20字内的运势建
50     议: "
51     try:
52         advice = model.invoke(prompt).content.strip()
53         return advice
54     except:
55         return "保持好心情, 好运自然来"

```

6.2.2 【构建 Plan-and-Execute 链】

接下来, 我们构建规划节点。规划节点负责生成步骤列表, 并且我们还增加了提取日期和判断是否询问明天运势两个辅助函数。

```

pae.py
1  # ===== State 定义 =====
2  class FortuneState(TypedDict):
3      messages: Annotated[list, add_messages]
4      user_info: dict          # birth_date, zodiac, chinese_zodiac, luck,
5                                advice
6      plan: List[str]          # 待执行计划
7      past_steps: List[tuple]  # 已执行步骤
8      final_answer: str

```



```

8     needs_replan: bool          # 是否需要重新规划
9
10    # ===== 辅助函数 =====
11    def extract_birth_date(text: str) -> str:
12        """提取日期, 支持: 1995年8月23日 / 1995-08-23 / 1995/8/23"""
13        # 中文格式: 1995年8月23日
14        chinese_pattern = r'(\d{4})年(\d{1,2})月(\d{1,2})日'
15        match = re.search(chinese_pattern, text)
16        if match:
17            year, month, day = match.groups()
18            return f"{year}-{int(month):02d}-{int(day):02d}"
19
20        # 标准格式: 1995-08-23 或 1995/8/23
21        standard_pattern = r'(\d{4})[-/](\d{1,2})[-/](\d{1,2})'
22        match = re.search(standard_pattern, text)
23        if match:
24            year, month, day = match.groups()
25            return f"{year}-{int(month):02d}-{int(day):02d}"
26
27        return ""
28
29    def is_asking_tomorrow(text: str) -> bool:
30        """判断是否询问明天运势"""
31        keywords = ["明天", "明日", "后天", "未来", "下次", "再算"]
32        return any(kw in text for kw in keywords)
33
34    def planner(state: FortuneState):
35        """
36        规划节点: 根据已有信息动态生成计划
37        - 第一次: 计算所有
38        - 以后: 只重新算运势和建议
39        """
40        user_info = state.get("user_info", {})
41        ask_tomorrow = state.get("ask_tomorrow", False)
42
43        plan = []
44
45        # 只有没有星座时才计算 (避免重复)
46        if not user_info.get("zodiac"):
47            plan.append("get_zodiac")
48            print("    [规划] 添加: get_zodiac")
49
50        if not user_info.get("chinese_zodiac"):
51            plan.append("get_chinese_zodiac")
52            print("    [规划] 添加: get_chinese_zodiac")
53
54        # 运势和建议每次都重新计算 (支持今天/明天)

```

```

55     plan.append("get_daily_luck")
56     plan.append("generate_advice")
57     print(f"    [规划] 添加: get_daily_luck (明天={ask_tomorrow})")
58     print(f"    [规划] 添加: generate_advice")
59
60     return {
61         "plan": plan,
62         "ask_tomorrow": ask_tomorrow,
63         "past_steps": [],
64         "needs_replan": False
65     }

```

然后我们继续增加执行节点，执行节点的作用是按照计划依次执行，调用对应的工具函数：

代码块

```

1  def executor(state: FortuneState):
2      """执行节点：单步执行"""
3      plan = state.get("plan", [])
4      user_info = state.get("user_info", {}).copy()
5      past_steps = state.get("past_steps", []).copy()
6      ask_tomorrow = state.get("ask_tomorrow", False)
7
8      if not plan:
9          return {"needs_replan": True} # 触发反思
10
11     step = plan[0]
12     result = ""
13
14     print(f"    [执行] {step}")
15
16     if step == "get_zodiac":
17         birth_date = user_info.get("birth_date")
18         result = get_zodiac.invoke({"birth_date": birth_date})
19         user_info["zodiac"] = result
20         print(f"    [结果] 星座: {result}")
21
22     elif step == "get_chinese_zodiac":
23         birth_date = user_info.get("birth_date")
24         result = get_chinese_zodiac.invoke({"birth_date": birth_date})
25         user_info["chinese_zodiac"] = result
26         print(f"    [结果] 生肖: {result}")
27
28     elif step == "get_daily_luck":
29         zodiac = user_info.get("zodiac", "未知")
30         day_offset = 1 if ask_tomorrow else 0

```

```

31         result = get_daily_luck.invoke({"zodiac": zodiac, "day_offset":
day_offset})
32         user_info["luck"] = result
33         print(f"    [结果] {result}")
34
35     elif step == "generate_advice":
36         zodiac = user_info.get("zodiac", "未知")
37         chinese_zodiac = user_info.get("chinese_zodiac", "未知")
38         luck = user_info.get("luck", "未知")
39         result = generate_advice.invoke({
40             "zodiac": zodiac,
41             "chinese_zodiac": chinese_zodiac,
42             "luck": luck
43         })
44         user_info["advice"] = result
45         print(f"    [结果] 建议: {result}")
46
47     past_steps.append((step, result))
48
49     return {
50         "plan": plan[1:],
51         "user_info": user_info,
52         "past_steps": past_steps,
53         "needs_replan": False
54     }

```

6.2.3 【条件路由】

接着，我们需要设置条件边步骤，该步骤的作用是决定继续执行下一步，还是进入反思或最终节点。

代码块

```

1
2  def route_after_collect(state: FortuneState) -> Literal["plan", "end"]:
3      """收集后路由：有生日则规划，否则结束等待"""
4      if state.get("final_answer"): # 缺少信息，直接返回答案
5          return "end"
6      return "plan"
7
8  def route_after_executor(state: FortuneState) -> Literal["execute", "reflect"]:
9      """执行后路由：还有步骤则继续，否则反思"""
10     if state.get("plan"):
11         return "execute"

```

```

12     return "reflect"
13
14 def route_after_reflector(state: FortuneState) -> Literal["execute",
    "finalize"]:
15     """反思后路由：需要补充则执行，否则总结"""
16     if state.get("plan"): # 反思后发现缺失，补充执行
17         return "execute"
18     return "finalize"

```

6.3 集成记忆与反思

6.3.1 【反思节点】

计划执行完后，我们让模型反思一下结果是否完整、合理。如果发现缺少关键信息，就重新规划。

代码块

```

1  def reflector(state: FortuneState):
2      """
3      反思节点：检查结果完整性
4      - 如果缺少关键字段，重新规划补充
5      """
6      user_info = state.get("user_info", {})
7      past_steps = state.get("past_steps", [])
8
9      print(f"    [反思] 检查完整性...")
10
11     # 检查必备字段
12     missing = []
13     if not user_info.get("zodiac"):
14         missing.append("get_zodiac")
15     if not user_info.get("chinese_zodiac"):
16         missing.append("get_chinese_zodiac")
17     if not user_info.get("luck"):
18         missing.append("get_daily_luck")
19     if not user_info.get("advice"):
20         missing.append("generate_advice")
21
22     if missing:
23         print(f"    [反思] 发现缺失: {missing}, 重新规划")
24         return {
25             "plan": missing,

```

```

26         "needs_replan": False # 补充执行, 不结束
27     }
28
29     print(f"    [反思] 检查通过, 结果完整")
30     return {"needs_replan": True} # 进入总结

```

注意：实际实现中，我们可以设计一个 `revise` 节点来处理修正，但为了简洁，这里直接在反思节点中重新规划缺失步骤。

6.3.2 【记忆管理】

LangGraph 的 `StateGraph` 配合 `checkpointer` 可以轻松实现多轮对话记忆。我们只需要在编译时传入一个检查点存储器，并在每次调用时使用相同的 `thread_id` 即可。这里提醒下，有关 LangGraph 框架我们会在后面专门讲解，还是老样子，大家通过更多的案例来获取编码感知，建立思维记忆即可，这样做的好处是我们可以削弱学习曲线，降低学习成本。

代码块

```

1  # 编译
2  checkpointer = InMemorySaver()
3  graph = StateGraph(FortuneState)
4  # ... 添加节点和边 ...
5  app = graph.compile(checkpointer=checkpointer)
6
7  # 使用同一个thread_id实现记忆
8  config = {"configurable": {"thread_id": "demo_user_001"}}

```

这样做的好处是，当用户问完‘我1995年8月23日出生，今天运势如何？’之后，再问‘那明天呢？’，机器人会记住之前的星座和生肖。

6.3.3 【构建完整图】

好了，整体编码完毕。现在，我们把所有节点连接起来，完成编码。

代码块

```

1  # ===== 节点函数 =====
2
3  def collect_info(state: FortuneState):
4      """

```

```

5     信息收集节点
6     - 提取生日，存入 user_info
7     - 如果已有生日，检查是否问明天
8     """
9     messages = state.get("messages", [])
10    user_info = state.get("user_info", {}).copy()
11    last_msg = messages[-1].content if messages else ""
12
13    # 尝试提取新生日
14    birth_date = extract_birth_date(last_msg)
15    if birth_date:
16        user_info["birth_date"] = birth_date
17        print(f"    [收集] 提取到生日: {birth_date}")
18
19    # 检查是否询问明天 (且已有生日)
20    ask_tomorrow = is_asking_tomorrow(last_msg)
21
22    # 判断是否有足够信息
23    if not user_info.get("birth_date"):
24        return {
25            "user_info": user_info,
26            "final_answer": "请提供您的出生日期（格式：YYYY-MM-DD），以便为您推算运
势。",
27            "needs_replan": False
28        }
29
30    # 已有生日，进入规划阶段
31    return {
32        "user_info": user_info,
33        "ask_tomorrow": ask_tomorrow,
34        "needs_replan": True
35    }
36
37    # 添加节点
38    graph.add_node("collect", collect_info)
39    graph.add_node("planner", planner)
40    graph.add_node("executor", executor)
41    graph.add_node("reflector", reflector)
42    graph.add_node("finalizer", finalizer)
43
44    # 边
45    graph.add_edge(START, "collect")
46
47    graph.add_conditional_edges(
48        "collect",
49        route_after_collect,
50        {"plan": "planner", "end": END}

```

```

51 )
52
53 graph.add_edge("planner", "executor")
54
55 graph.add_conditional_edges(
56     "executor",
57     route_after_executor,
58     {"execute": "executor", "reflect": "reflector"}
59 )
60
61 graph.add_conditional_edges(
62     "reflector",
63     route_after_reflector,
64     {"execute": "executor", "finalize": "finalizer"}
65 )
66
67 graph.add_edge("finalizer", END)
68
69 app = graph.compile(checkpointer=checkpointer)

```

6.4 演示交互

6.4.1 【模拟对话】

代码编写完毕后，下面我们执行代码。代码执行成功，我们来模拟用户与命理机器人的对话。假设用户第一次问：

代码块

```
1 你：我1995年8月23日出生，今天运势如何？
```

机器人内部执行流程：

1. `collect_info` 提取到出生日期 `1995-08-23`，存入 `user_info`。
2. `planner` 生成计划：`["get_zodiac", "get_chinese_zodiac", "get_daily_luck", "generate_advice"]`。
3. `executor` 依次执行：
 - `get_zodiac("1995-08-23")` → 处女座

- `get_chinese_zodiac("1995-08-23")` → 猪
 - `get_daily_luck("处女座")` → 中吉
 - `generate_advice` 调用模型 → “注意人际关系，今天适合团队合作。”
4. `reflector` 检查结果：包含星座、生肖、运势、建议，合格。
 5. `finalizer` 输出最终回答。

最终输出：

```
=====
🧙 智能命理机器人 - 演示模式
=====

【场景1】第一次询问，完整推算
-----
👤 你：我1995年8月23日出生，今天运势如何？
[收集] 提取到生日：1995-08-23
[规划] 添加：get_zodiac
[规划] 添加：get_chinese_zodiac
[规划] 添加：get_daily_luck (明天=False)
[规划] 添加：generate_advice
[执行] get_zodiac
[结果] 星座：处女座
[执行] get_chinese_zodiac
[结果] 生肖：猪
[执行] get_daily_luck
[结果] 今日运势：平
[执行] generate_advice
[结果] 建议：保持平和的心态，处理好人际关系，生活会更加顺心如意。
[反思] 检查完整性...
[反思] 检查通过，结果完整
[总结] 生成最终回答

🧙 助手：根据您的出生日期1995-08-23，您的星座是处女座，生肖是猪。今日运势：平。建议：保持平和的心态，处理好人际关系，生活会更加顺心如意。
```

6.4.2 【多轮对话与记忆】

接下来，用户继续问：

```
代码块

1  你：那明天的运势呢？
```

因为使用了同一个 `thread_id`，机器人记住了之前的星座和生肖。它会重新执行 `get_daily_luck`（因为运势每天不同）和 `generate_advice`，但不会重复计算星座和生肖。程序接着会输出：

【场景2】多轮对话，询问明天运势

你：那明天的运势呢？

助手：根据您的出生日期1995-08-23，您的星座是处女座，生肖是猪。今日运势：平。建议：保持平和的心态，处理好人际关系，生活会更加顺心如意。

【记忆状态】

生日：1995-08-23

星座：处女座（已记忆，下次不再计算）

生肖：猪（已记忆，下次不再计算）

6.4.3 【反思机制的效果】

但如果机器人因为某些原因（比如模型调用失败）没有生成建议，反思节点会检测到缺少‘建议’，然后重新规划并执行 `generate_advice` 步骤，确保回答完整。

如果用户没有提供出生日期，`collect_info` 节点会主动询问：

代码块

```
1 你：帮我算算运势
```

程序接着执行就会输出：

【场景3】缺少信息，主动询问

你：帮我算算运势

助手：请提供您的出生日期（格式：YYYY-MM-DD），以便为您推算运势。

你：2000年1月1日

[收集] 提取到生日：2000-01-01

助手：请提供您的出生日期（格式：YYYY-MM-DD），以便为您推算运势。

这就是整个命理机器人的动态交互能力。

下面，我们再做个总结：通过这个命理机器人的实战，我们成功融合了 ReAct、Plan-and-Execute、Self-Ask 和 Reflection 四种认知框架。我们使用了 LangGraph 框架，从这个样例你可以清晰看到看到，LangGraph 框架提供了强大的状态管理和流程控制能力，可以让我们能够灵活地编排这些模式。

第七部分：课程总结与作业

7.1 核心要点回顾

7.1.1 【四种认知框架总结】

好，今天的课程内容已经结束了，知识非常丰富，我们一口气学习了四种 Agent 认知框架。现在，我再来带大家快速回顾一下，让这些知识点在你脑子里形成一张清晰的网络。

第一种：ReAct模式

就是思考-行动-观察的循环。模型每一步都在想‘我现在知道什么、下一步该做什么’，然后动手，观察结果，再继续。它的核心价值是**动态决策**——能够根据环境反馈实时调整策略。适合客服、研究助手等需要灵活应对的任务。

第二种：Plan-and-Execute

先规划后执行。模型先生成一份步骤清单，然后按顺序执行，执行过程中可以复用工具函数而不必每次都调用大模型。它的核心价值是**高效稳定**——成本低、速度快、可预测。适合数据流水线、批量报告等结构化任务。

第三种：Self-Ask

自问自答，回路验证。模型把大问题拆解成多个子问题，逐一检索答案，最后综合。它的核心价值是**知识检索**——确保每一步都有据可查，避免模型编造。适合事实问答、多跳推理。

第四种：Thinking and Self-Reflection

自我评价与修正。模型对自己的输出进行检查，发现不足就修改，可以迭代多轮。它的核心价值是**提升可靠性**——让Agent从‘闷头做事’变成‘会自我改进’。适合代码生成、数据分析、规划等高价值任务。

7.1.2 【模式之间的关系与组合】

需要注意的是，这四种模式并不是孤立的，它们可以组合使用，就像乐高积木一样。一个成熟的 Agent 往往同时具备多种认知能力。这里举几个组合的例子。

1. **ReAct + Self-Ask**：在动态交互中，当模型不确定时，先自问需要查什么，然后调用搜索工具。这是最常用的组合。

2. **Plan-and-Execute + Reflection**：先规划执行，执行完后反思结果是否达标，如果不达标就重规划或修正。这在企业级应用中非常常见。
3. **ReAct + Plan-and-Execute**：先用 ReAct 动态收集信息、澄清需求，然后切换到 Plan-and-Execute 高效执行固定流程。这是‘混合模式’的典型代表。

在今天的命理机器人实战中，我们实际上融合了这四种模式：用 ReAct 处理信息收集，用 Plan-and-Execute 处理计算步骤，用 Self-Ask 检查遗漏，用 Reflection 确保答案完整。

7.1.3 【总结金句】

在课程的最后，这里送给大家一句话，作为今天课程的结语：

Agent 的智能，不在于它能调用多少工具，而在于它如何思考、规划、验证和反思。

大家掌握了这四种认知框架，就从‘会写Agent的人’进化成了‘会设计Agent大脑的人’。这是构建高级智能体的内功，也是我们后续学习多 Agent 系统、LangGraph 高级特性的基础。

好了，本节课核心要点就回顾到这里。

7.2 课后作业

7.2.1 【必做作业】

接下来还是老习惯，提供几个课后作业，大家实际动手实操下，巩固今天所学。

1. **运行今天的代码，实现命理机器人**，包含星座、生肖、运势查询，并能连续对话。要求：能够从用户输入中提取出生日期，如果用户没提供则主动询问；能够记住之前的星座和生肖，支持‘那明天呢？’这样的追问。
2. **尝试将命理机器人的核心计算部分改为 Plan-and-Execute 模式**，并记录执行过程。也就是说，不要用 ReAct 一步步调用，而是先生成一个计划（比如‘先算星座，再算生肖，再算运势’），然后按计划执行。对比两种模式的代码复杂度和运行效率。
3. **为命理机器人增加一个简单的反思机制**：在输出最终答案前，检查是否包含了星座、生肖、运势三项。如果缺少任意一项，自动重新生成或补充。

这三项完成后，相信大家就能熟练运用今天学到的认知框架了。这对以后的 Agent 智能体开发打下非常好的基础。

附录：教学资源

推荐阅读

- ReAct论文: [ReAct: Synergizing Reasoning and Acting in Language Models](#)
- Plan-and-Execute论文: [Plan-and-Solve Prompting](#)
- Self-Ask论文: [Self-Ask: Measuring and Narrowing the Compositionality Gap in Language Models](#)